

# Non-Automated Automated Testing

Bob Fornal



## Entrepreneur

Code-squid provides solid, in-depth frontend training that is supported with real-world code projects. Blessed husband and proud father of two.

<https://code-squid.com>



## Senior Solutions Developer Leading EDJE, Inc.

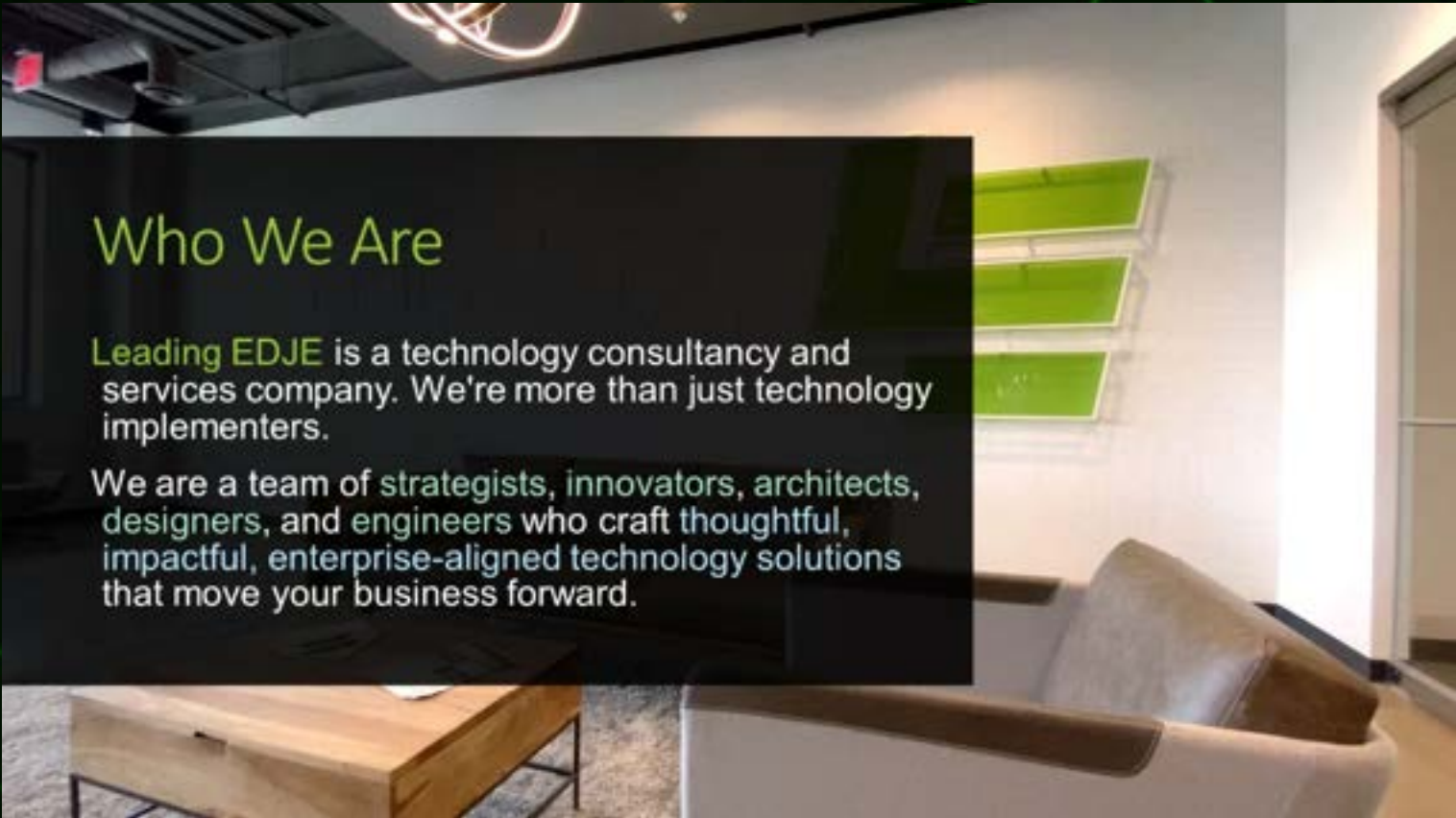
Passionate about learning, testing, mentoring, speaking, and personal growth.

<https://leadingedje.com>



1 / 12





## Who We Are

**Leading EDJE** is a technology consultancy and services company. We're more than just technology implementers.

We are a team of strategists, innovators, architects, designers, and engineers who craft thoughtful, impactful, enterprise-aligned technology solutions that move your business forward.

# What is Manual Testing?

## Manual Testing

The process of testing software applications by hand, **without the use of automated tools**. Testers manually execute test cases, simulating how an end-user would interact with the software, and **verify the results against expected outcomes**. This method is crucial for identifying defects and ensuring software quality before release.

### Key Aspects:

- No Automation
- Exploratory Testing
- Usability Testing
- Regression Testing
- Versatility



# What is Automated Testing?

## Automated Testing

A software testing methodology that utilizes specialized tools and scripts to execute test cases and compare **actual results against expected outcomes**, with **minimal or no human intervention** during the execution phase. This contrasts with **manual testing**, where human testers manually interact with the software to identify defects.

### Core Principles:

- Test Script Creation
- Automated Execution
- Results Analysis
  
- Unit Tests
- Integration Tests
- End-to-End Tests





# What is Non-Automated Automated Testing?

## Non-Automated Automated Testing

This is a term that I started using when a production deployment went sideways for an "odd reason."

### Production deployments often use hybrid approaches:

- Automated tests run to verify key functionality.
- Humans perform contextual verification/checks that automation cannot fully capture.

## Definition:

Automated test tools (or scripts) are used to initiate and carry out predefined checks in **the live production environment**.

The execution is complemented by manual steps that cannot be fully automated:

- Triggering jobs.
- Validating test data
- Interpreting results.
- Performing exploratory checks.

# What is Non-Automated Automated Testing?

## Non-Automated Automated Testing

This is a term that I started using when a production deployment went sideways for an "odd reason."

### Production deployments often use hybrid approaches:

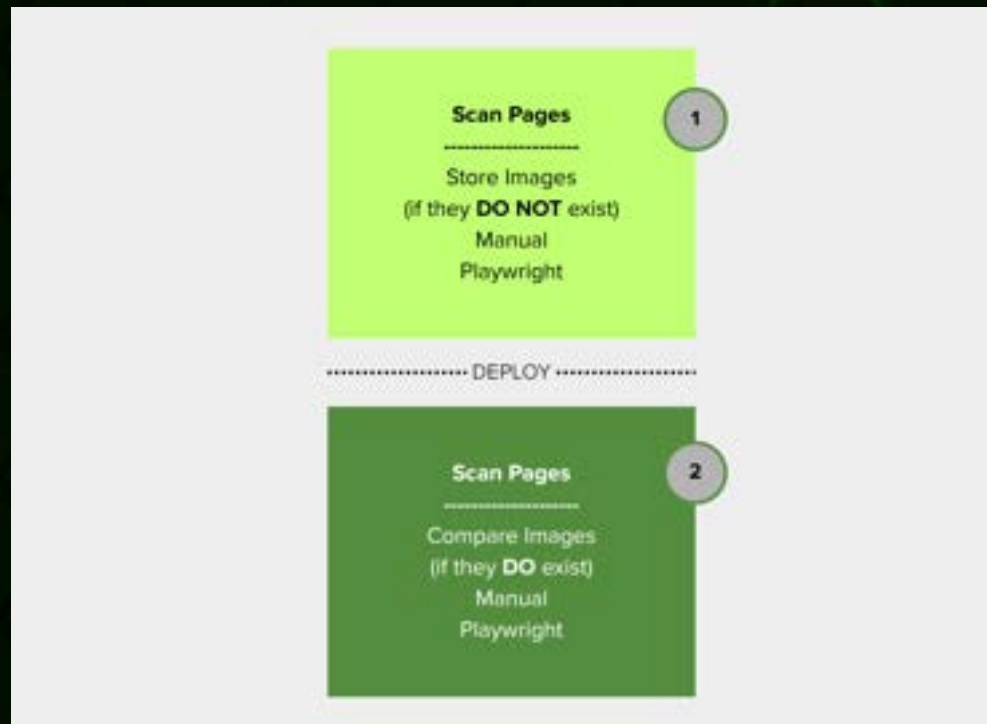
- Automated tests run to verify key functionality.
- Humans perform contextual verification/checks that automation cannot fully capture.

## Automation Ensures:

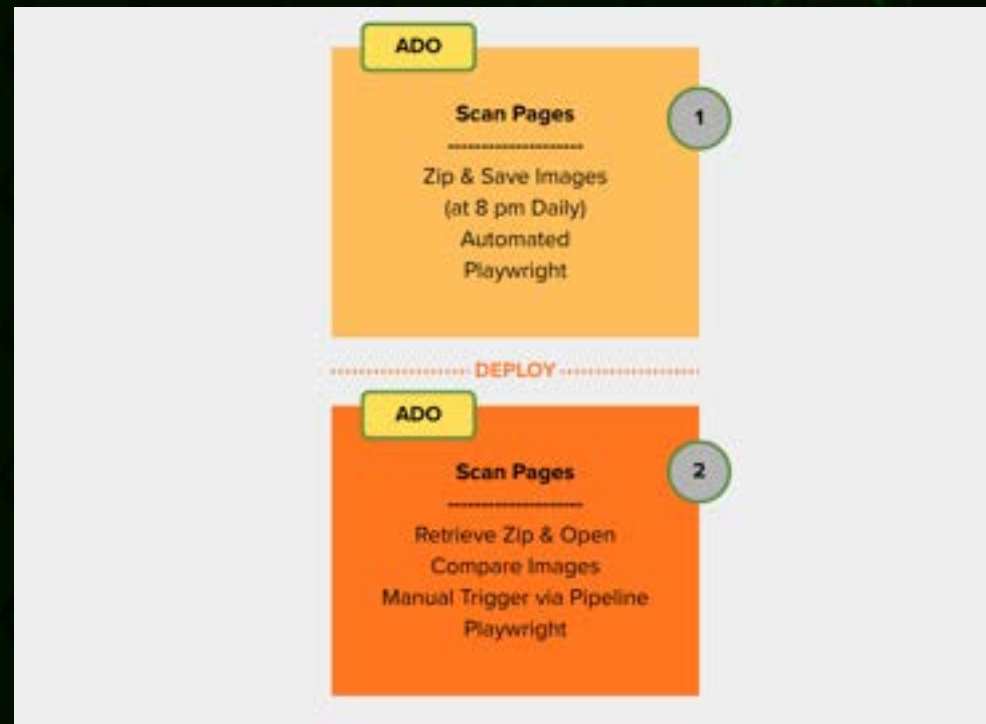
Consistent, repeatable coverage of **known requirements**, while human testers provide **contextual judgment**, visual validation, and **decision-making** before, during, or after the automated tests are run.

The combined approach aims to **quickly detect production issues** and **confirm deployment success** while **accounting for real-world conditions** automation alone may miss.

# Functional VRT



# Functional Pipeline & VRT





# Why use Non-Automated Automated Testing?

## Concise Definition

Non-automated automated testing (in production) is a **hybrid testing approach** where automated tools execute predefined test actions **against a live deployment**, but human testers **manually initiate, observe, interpret, and supplement these actions** with additional checks to **validate real-world behavior** before finalizing the release as successful.

## Why use it?

- Automation is fast and repeatable, but blind to qualitative, visual, and contextual nuances.
- Manual testing is adaptable and insightful, especially when production data or environments behave differently than staging.
- In production deployments, risk tolerance is low, so blending automation with human oversight reduces false negatives and false positives.

## Common Risks & Failure

1. Human dependency & Variability: Missed defects, slower execution, unpredictable quality sign-offs.
2. Misinterpretation of automated results: false positives or negatives.
3. Automation drift: false sense of coverage; critical paths go untested.
4. Critical-path blind spots: Undetected defects in the most important user flows.



